Complete computations for the mix criteria

Human Compression project

Author edition; pending independent review

Roles of the complete files

The GAP file supplies the finite certificate for the literal eleven-cell and simplex markings used in the coprime-label obstruction. Its group arithmetic is a retained computational premise. The subsequent product argument is written in full in the application.

The Python file is bounded validation of the complete six-coordinate argument. It checks all input flag intersections and actual automorphisms, and witnesses the failure in their mix. The displayed coordinate proof establishes that example independently of an extrapolation from test cases. Neither program proves the general path characterization or all source statements. The replay driver and the two-declaration Lean source are inventoried separately.

Required finite certificate

```
# Literal finite certificate for the coprime-label counterexample.
# GAP subgroup arithmetic is the retained computational kernel.
Need := function(ok, msg)
  if not ok then Error(msg); fi;
end;
CheckIP := function(marks)
  local subs, mask, i, j, common, mask2;
  subs := [];
  for mask in [0..15] do
    Add(subs, Group(Filtered(marks, x -> QuoInt(mask,2^(Position(marks,x)-1)) mod 2 = 1), ()));
  od;
  for mask in [0..15] do
    for mask2 in [0..15] do
      common := Sum(Filtered([0..3], i -> QuoInt(mask,2^i) mod 2=1 and QuoInt(mask2,2^i) mod 2=1), i -> 2^i);
      Need(Intersection(subs[mask+1],subs[mask2+1])=subs[common+1], "input intersection property");
    od;
  od;
  return subs;
end;
a := [(1,2)(3,5)(4,7)(6,8), (1,3)(2,4)(5,7)(6,9),
      (3,6)(4,7)(5,8)(9,10), (1,3)(2,5)(4,7)(10,11)];;
b := [(12,13),(13,14),(14,15),(15,16)];;
for marks in [a,b] do
  Need(ForAll(marks, x -> Order(x)=2), "nonidentity involutions");
  for i in [1..4] do
    for j in [i+2..4] do
      if j<=4 then Need(marks[i]*marks[j]=marks[j]*marks[i], "string commutation"); fi;
    od;
  od;
od;
G := Group(a);; S := Group(b);;
Need(Size(G)=660 and IsSimpleGroup(G), "literal simple 660-group");
Need(Size(S)=120, "literal simplex S5");
Need(List([1..3], i -> Order(a[i]*a[i+1]))=[3,5,3], "first type");
Need(List([1..3], i -> Order(b[i]*b[i+1]))=[3,3,3], "second type");
sa:=CheckIP(a);;sb:=CheckIP(b);;
Need(Size(sa[8])=60 and Size(sa[15])=60 and IsSimpleGroup(sa[8]) and IsSimpleGroup(sa[15]), "first A5 ends");
Need(Size(sb[8])=24 and Size(sb[15])=24, "second S4 ends");
Need(ForAll(b,x->SignPerm(x)=-1), "second orientation character");
m:=List([1..4],i->a[i]*b[i]);;
M:=Group(m);;A:=Group(m{[1..3]});;B:=Group(m{[2..4]});;C:=Group(m{[2,3]});;
```

```
Need([Size(M),Size(A),Size(B),Size(C),Size(Intersection(A,B))]=[79200,1440,1440,30,60], "exact mixed orders");
Need(a[2] in A and a[2] in B and not(a[2] in C), "literal mixed intersection witness");
Print("COPRIME_CERTIFICATE_PASS\n");
Print("input_IP_checks=512; input_orders=660,120; types=3,5,3/3,3,3\n");
Print("mixed_orders=79200,1440,1440,30,60; witness=a1; second_orientable=true\n");
QUIT_GAP(0);
```

Bounded validation of the six-coordinate construction

```
#!/usr/bin/env python3
"""Complete finite validation of the written six-coordinate counterexample.
No source text, exploratory program or external algebra package is imported.
"""
import argparse, itertools, json
from pathlib import Path

def need(p, message):
    if not p:
        raise ValueError(message)

def product(g,h):
    return tuple(g[h[i]] for i in range(len(g)))

def inverse(g):
    return tuple(g.index(i) for i in range(len(g)))

def power(g,n):
    h=tuple(range(len(g)))
    for _ in range(n): h=product(h,g)
    return h

def closure(gens,identity):
    out=[identity]; seen={identity}
    for g in out:
        for s in gens:
            h=product(g,s)
            if h not in seen: seen.add(h);out.append(h)
    return out

def rank(rows,prime=3):
    a=[list(r) for r in rows];i=0
    for j in range(len(a[0])):
        k=next((k for k in range(i,len(a)) if a[k][j]%prime),None)
        if k is None: continue
        a[i],a[k]=a[k],a[i];m=pow(a[i][j],-1,prime)
        a[i]=[m*x%prime for x in a[i]]
        for k in range(len(a)):
            if k!=i:
                m=a[k][j];a[k]=[(x-m*y)%prime for x,y in zip(a[k],a[i])]
        i+=1
    return i

def span(rows):
    z=(0,)*len(rows[0]);out={z}
    for r in rows:
        out={tuple((v[i]+k*r[i])%3 for i in range(len(z))) for v in out for k in range(3)}
    return out

def component_data(adj):
    result={}
    for mask in range(16):
        remaining=set(range(len(adj)));bits=[];index=[-1]*len(adj)
        while remaining:
            root=min(remaining);remaining.remove(root);q=[root];members={root}
            for v in q:
                for i in range(4):
                    if mask>>i&1:
                        w=adj[v][i]
                        if w not in members: members.add(w);q.append(w);remaining.remove(w)
            number=len(bits);b=sum(1<<v for v in members)
            for v in members:index[v]=number
            bits.append(b)
        result[mask]=(bits,index)
```

```
    return result

def at(data,mask,v):
    bits,index=data[mask];return bits[index[v]]

def all_ip(data,vertices):
    count=0
    for v in range(vertices):
        for a in range(16):
            for b in range(16):
                need(at(data,a,v)&at(data,b,v)==at(data,a&b,v),f'input IP: {v},{a},{b}')
                count+=1
    return count

def calculate():
    S=list(itertools.permutations(range(5)));sid={g:i for i,g in enumerate(S)};e=tuple(range(5))
    simple=[]
    for i in range(4):
        s=list(e);s[i],s[i+1]=s[i+1],s[i];simple.append(tuple(s))
    cycles=[(0,1,2,3,4),(0,1,2,4,3),(0,1,3,4,2),(0,1,3,2,4),(0,1,4,3,2),(0,1,4,2,3)]
    H=[]
    for cyc in cycles:
        g=list(e)
        for i in range(5):g[cyc[i]]=cyc[(i+1)%5]
        H.append(frozenset(power(tuple(g),j) for j in range(5)))
    need(len(set(H))==6,'six distinct Sylow groups')
    pi={}
    for g in S:
        ig=inverse(g)
        pi[g]=tuple(H.index(frozenset(product(product(g,h),ig) for h in K)) for K in H)
    expected=[(4,2,1,5,0,3),(5,3,4,1,2,0),(3,2,1,0,5,4),(1,0,4,5,2,3)]
    need([pi[s] for s in simple]==expected,'literal six-coordinate action')
    def act(g,v):
        out=[0]*6
        for i,j in enumerate(pi[g]):out[j]=v[i]
        return tuple(out)
    zero=(0,)*6;u=[zero,(2,2,0,1,0,1),zero,zero]
    R=[(u[i],simple[i]) for i in range(4)]
    def mult(x,y):
        v,g=x;w,h=y;aw=act(g,w)
        return tuple((v[i]+aw[i])%3 for i in range(6)),product(g,h)
    def word(indices):
        x=(zero,e)
        for i in indices:x=mult(x,R[i])
        return x
    for i in range(4): need(word([i,i])==(zero,e),'involution')
    for i,j in [(0,2),(0,3),(1,3)]:need(word([i,j,i,j])==(zero,e),'distant square')
    v=(1,1,2,2,1,2)
    need(word([1,2]*3)==(v,e),'middle cube')
    need(word([0,1]*3)==word([2,3]*3)==(zero,e),'outer cubes')
    A0=closure(simple[:3],e);B0=closure(simple[1:],e)
    VA=span([act(g,v) for g in A0]);VB=span([act(g,v) for g in B0]);VC=span([v])
    expectedA=span([(1,0,0,0,0,2),(0,1,2,0,0,0),(0,0,0,1,2,0)])
    expectedB=span([(1,0,0,2,0,0),(0,1,0,0,0,2),(0,0,1,0,2,0)])
    need(VA==expectedA and VB==expectedB,'actual end translation spaces')
    need(rank([act(g,v) for g in S])==5,'whole augmentation space')
    def factor(h):
        return [[3*sid[product(g,simple[i])]+(k+act(g,u[i])[h])%3 for i in range(4)] for g in S for k in range(3)]
    reports=[]
    for h in [0,1]:
        adj=factor(h);data=component_data(adj)
        need(len(data[15][0])==1,'connected factor')
        count=all_ip(data,360)
        K=[g for g in S if pi[g][h]==h]
        need(len(K)==20,'Sylow normalizer order')
        need({(g[0],g[4]) for g in K}=={(a,b) for a in range(5) for b in range(5) if a!=b},'sharp two-transitivity')
        # Verify all displayed normalizer lifts and all three deck translations.
        for a in K:
            for t in range(3):
                lift=[3*sid[product(a,g)]+(k+t)%3 for g in S for k in range(3)]
                for q in range(360):
                    for i in range(4):need(lift[adj[q][i]]==adj[lift[q]][i],'actual colored automorphism lift')
        local={str(m):sorted({b.bit_count() for b in data[m][0]}) for m in [7,14,6]}
        need(local=={'7':[72],'14':[72],'6':[18]},'all uniform local sizes')
        reports.append({'coordinate':h,'flags':360,'all_subset_ip_checks':count,'normalizer_lifts':60,'local_sizes':local
        ,'middle_components':len(data[6][0])})
```

```
    mix=[[9*sid[product(g,simple[i])]+(k+act(g,u[i])[0])%3+3*((l+act(g,u[i])[1])%3) for i in range(4)] for g in S for l
    in range(3) for k in range(3)]
    data=component_data(mix);need(len(data[15][0])==1,'connected mix')
    for q in range(1080):
        for end in [7,14]:
            for a in range(16):
                if a&~end:continue
                for b in range(16):
                    if b&~end:continue
                    need(at(data,a,q)&at(data,b,q)==at(data,a&b,q),'all mixed end IP')
    gstar=product(product(simple[1],simple[2]),simple[3]);good=9*sid[gstar];bad=0
    need(good==81,'literal good base')
    def sizes(q):return [at(data,m,q).bit_count() for m in [7,14,6]]+[(at(data,7,q)&at(data,14,q)).bit_count()]
    need(sizes(good)==[72,216,18,18] and sizes(bad)==[216,216,18,54],'good and bad root intersections')
    def source_fiber(q,m):
        first=(q//9,q%3);bits=at(data,m,q)
        return {((w%9)//3-q%9//3)%3 for w in range(1080) if bits>>w&1 and (w//9,w%3)==first}
    need([source_fiber(good,m) for m in [7,14,6]]==[{0},{0,1,2},{0}],'literal lower variance base fibers')
    c=[1,2]*3;ha=[2,3];h_inv=list(reversed(ha));c_inv=list(reversed(c))
    wa=(c+[0]+c+[0])*2;wb=(c+ha+c_inv+h_inv)*2
    def walk(q,w):
        for i in w:q=mix[q][i]
        return q
    need(walk(0,wa)==walk(0,wb)==1,'two explicit path words')
    need(not(at(data,6,0)>>1&1),'witness not in the middle component')
    return {'status':'PASS','role':'bounded validation of complete written proof; no all-rank
    extrapolation','factor_reports':reports,'group_order':29160,'mix_flags':1080,'good_base':good,'bad_base':bad,'good_si
    zes':sizes(good),'bad_sizes':sizes(bad),'explicit_words':{'facet':wa,'vertex':wb},'counterexample_targets':['F005
    Corollary 6.4','F005 Theorem 6.7'],'qualified_targets':{'F005 Corollary 6.6':'operational middle-color-component
    transitivity; printed parenthetical chain type corrected','F005 Proposition 6.8':'same medial-section convention'}}

def main():
    p=argparse.ArgumentParser();p.add_argument('--out',type=Path,required=True);a=p.parse_args()
    r=calculate();a.out.parent.mkdir(parents=True,exist_ok=True);a.out.write_text(json.dumps(r,indent=2)+'\n');print(json
    .dumps({'status':r['status'],'factors':2,'flags_each':360,'mix_flags':1080}))
if __name__=='__main__':main()
```